

04. 분류 - Part1

01. 분류(Classification)의 개요

▼ 분류는 다양한 머신러닝 알고리즘으로 구현 가능

베이즈(Bayes) 통계와 생성 모델에 기반한 나이브 베이즈(Naive Bayes)

독립변수와 종속변수의 선형 관계성에 기반한 로지스틱 회귀

데이터 균일도에 따른 규칙 기반의 결정 트리

개별 클래스 간의 최대 분류 마진을 효과적으로 찾아주는 서포트 벡터 머신

근접 거리를 기준으로 하는 최소 근접 알고리즘

심층 연결 기반의 신경망

서로 다른(또는 같은) 머신러닝 알고리즘을 결합한 앙상블

⇒ 이 중 앙상블 방법(Ensemble Method)에 대해 집중적으로 알아보자!

앙상블 방법(Ensemble Method)

- 서로 다른(또는 같은) 머신러닝 알고리즘을 결합함 - 대부분 동일한 알고리즘을 결합
- 정형 데이터의 예측 분석 영역에서 매우 높은 예측 성능으로 많이 쓰임
- 일반적으로 배깅(Bagging)과 부스팅(Boosting) 방식으로 나뉨

배깅(Bagging)	부스팅(Boosting)
랜덤 포레스트 (Random Forest) - 뛰어난 예측 성능, 상대적으로 빠른 수행 시간, 유연성	그래디언트 부스팅(Gradient Boosting) - 뛰어난 예측 성능, 수행 시간 오래걸림 ⇒ XgBoost, LightGBM 등 예측 성능 향상 및 수행 시간 단축

- 앙상블의 기본 알고리즘으로 일반적으로 사용하는 것은 결정 트리

? 결정 트리란...?

- 매우 쉽고 유연하게 적용될 수 있는 알고리즘

- 데이터의 스케일링이나 정규화 등의 사전 가공 영향이 매우 적음
- 예측 성능을 향상시키기 위해 복잡한 규칙 구조를 가짐 → 과적합 (overfitting)이 발생해 반대로 예측 성능이 저하 될 우려가 있음

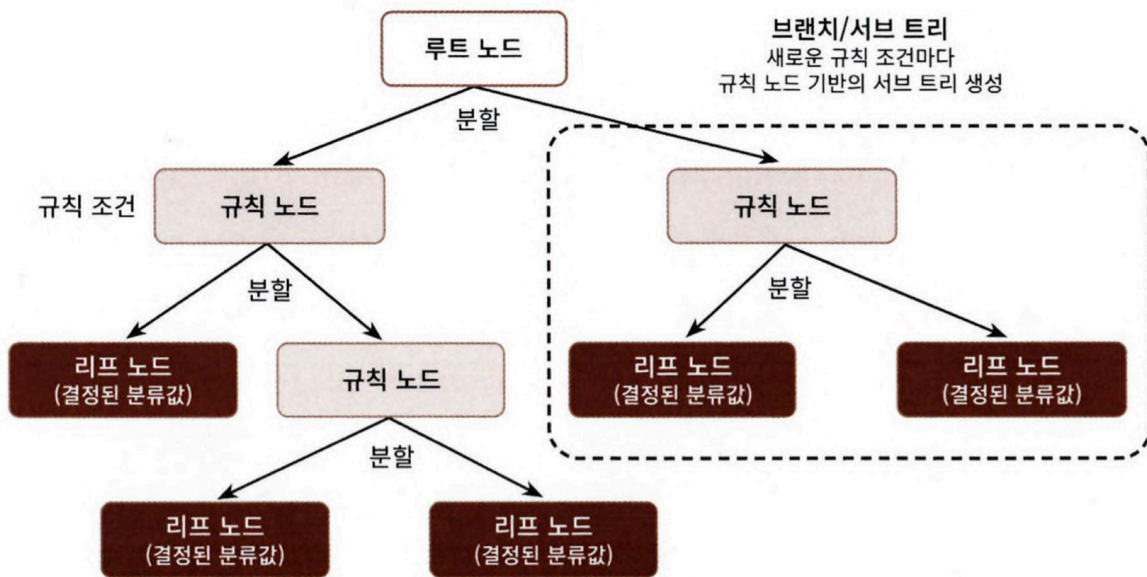
⇒ 이러한 단점이 앙상블 기법에서는 오히려 장점으로 작용!

- 앙상블은 매우 많은 여러 개의 예측 성능이 상대적으로 떨어지는 학습 알고리즘을 결합해 확률적으로 보완과 오류가 발생한 부분에 대한 가중치를 계속 업데이트 하면서 예측 성능을 향상 → 이 때, 결정 트리가 좋은 약한 학습기가 됨

02. 결정 트리(Decision Tree)

- 데이터에 있는 규칙을 학습을 통해 자동으로 찾아내 트리 기반의 분류 규칙을 만듦

⇒ 데이터의 어떤 기준을 바탕으로 규칙을 만들어야 가장 효율적인 분류가 될 것인가가 알고리즘의 성능을 크게 좌우함



규칙 노트(Decision Node) - 규칙 조건이 되는 것

리프 노트(Leaf Node) - 결정된 클래스 값

서브 트리(Sub Tree) - 새로운 규칙 조건마다 생성됨

데이터 세트에 피처가 있고 이러한 피처가 결합해 규칙 조건을 만들 때마다 규칙 노드가 만들어짐.

많은 규칙이 있다 = 분류를 결정하는 방식이 복잡하다

⇒ 과적합으로 이어지기 쉬움!!

즉, 트리의 깊이(depth)가 깊어질수록 결정 트리의 예측 성능이 저하될 가능성이 높다

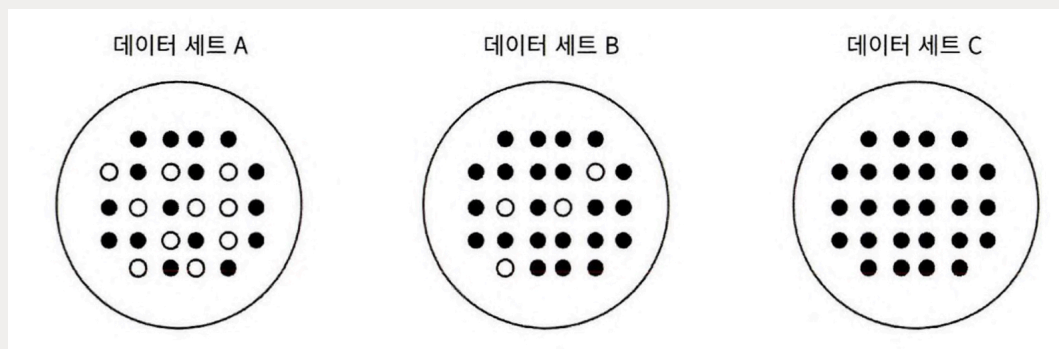
가능한 한 적은 결정 노드로 높은 예측 정확도를 가지려면

→ 데이터를 분류할 때 최대한 많은 데이터 세트가 해당 분류에 속할 수 있도록 결정 노드의 규칙이 정해져야 함

→ 어떻게 트리를 분할(split)할 것인가가 중요

→ 최대한 균일한 데이터 세트를 구성할 수 있도록 분할하는 것이 필요

? 균일한 데이터 세트란...?



균일도: $C > B > A$

⇒ 데이터 세트의 균일도는 데이터를 구분하는 데 필요한 정보의 양에 영향을 미침

ex) 눈을 가린 채 데이터 세트C에서 하나의 데이터를 뽑았을 때, 데이터에 대한 별다른 정보 없이도 '검은 공'이라고 쉽게 예측 가능. 그러나 A의 경우는 상대적으로 혼잡도가 높고 균일도가 낮기 때문에 같은 조건에서 데이터를 판단하는 데 있어 더 많은 정보가 필요

결정 노드는 정보 균일도가 높은 데이터 세트를 먼저 선택할 수 있도록 규칙 조건을 만들

→ 정보 균일도가 데이터 세트로 쪼개질 수 있도록 조건을 찾아 서브 데이터 세트를 만들고, 다시 이 서브 데이터 세트에서 균일도가 높은 자식 데이터 세트를 쪼개는 방식을 자식 트리로 내려가면서 반복하는 방식으로 데이터 값을 예측



균일도를 측정하는 법...?

정보 이득(Information Gain) 지수

정보 이득은 엔트로피라는 개념을 기반으로 함. 엔트로피는 주어진 데이터 집합의 혼잡도를 의미 - 서로 다른 값이 섞여 있으면 엔트로피가 높고, 같은 값이 섞여 있으면 엔트로피가 낮음.

정보 이득 지수 = 1 - 엔트로피 지수

→ 정보 이득이 높은 속성을 기준으로 분할

지니 계수

지니 계수는 원래 경제학에서 불평등 지수를 나타낼 때 사용하는 계수.

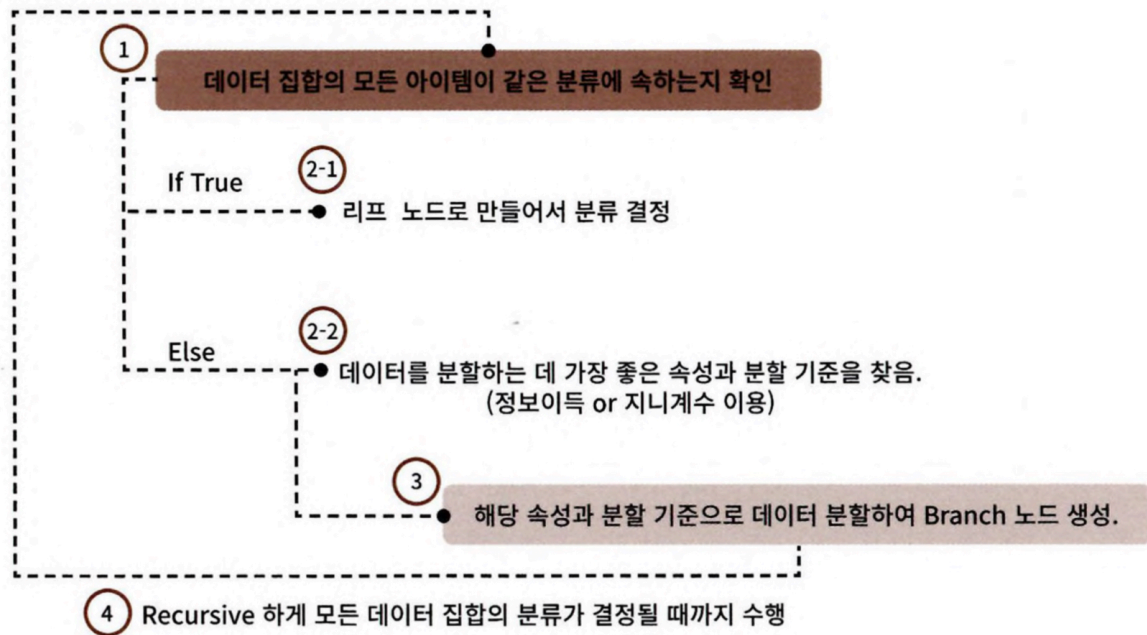
0(가장 평등) ← ————— → 1(가장 불평등)

머신러닝에 적용될 때는 지니 계수가 낮을수록 데이터 균일도가 높은 것으로 해석

→ 지니 계수가 낮은 속성을 기준으로 분할

결정 트리 알고리즘을 사이킷런에서 구현한 DecisionTreeClassifier은 기본으로 지니 계수를 이용해 데이터 세트를 분할

- 결정 트리의 일반적인 알고리즘은 데이터 세트를 분할하는 데 가장 좋은 조건 즉, 정보 이득이 높거나 지니 계수가 낮은 조건을 찾아서 자식 트리 노드에 걸쳐 반복적으로 분할한 뒤, 데이터가 모두 특정 분류에 속하게 되면 분할을 멈추고 분류를 결정



결정 트리 모델의 특징

장점

1. 쉽고 직관적이다
2. 피처의 스케일링이나 정규화 등의 사전 가공 영향도가 크지 않다

단점

1. 과적합으로 알고리즘 성능이 떨어진 다. 이를 극복하기 위해 트리의 크기를 사전에 제한하는 튜닝 필요하다

결정 트리 파라미터

사이킷런은 결정 트리 알고리즘을 구현한 `DecisionTreeClassifier` 클래스(분류)와 `DecisionTreeRegressor` 클래스(회귀)를 제공

사이킷런의 결정 트리 구현은 CART(Classification And Regression Trees) 알고리즘 기반

DecisionTreeClassifier와 DecisionTreeRegressor 모두 파라미터는 다음과 동일한 파라미터를 사용

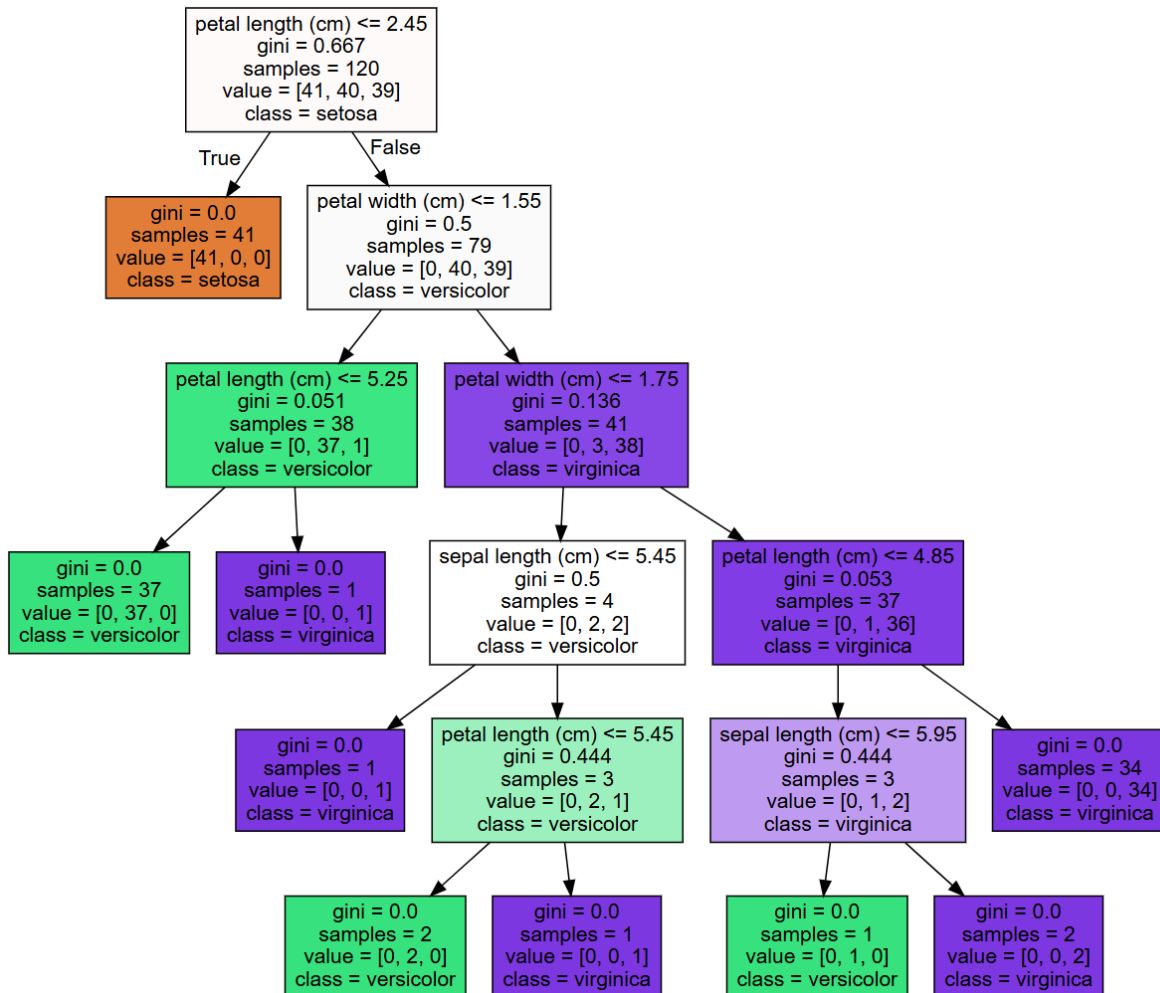
파라미터 명	설명
min_samples_split	• 노드를 분할하기 위한 최소한의 샘플 데이터 수로 과적합을 제어하는 데 사용됨. • 디폴트는 2이고 작게 설정할수록 분할되는 노드가 많아져서 과적합 가능성 증가
min_samples_leaf	• 분할이 될 경우 왼쪽과 오른쪽의 브랜치 노드에서 가져야 할 최소한의 샘플 데이터 수 • 큰 값으로 설정될수록, 분할될 경우 왼쪽과 오른쪽의 브랜치 노드에서 가져야 할 최소한의 샘플 데이터 수 조건을 만족시키기가 어려우므로 노드 분할을 상대적으로 덜 수행함. • min_samples_split와 유사하게 과적합 제어 용도, 그러나 비대칭적(imbalanced) 데이터의 경우 특정 클래스의 데이터가 극도로 작을 수 있으므로 이 경우는 작게 설정 필요.
max_features	• 최적의 분할을 위해 고려할 최대 피쳐 개수. 디폴트는 None으로 데이터 세트의 모든 피쳐를 사용해 분할 수행. • int 형으로 지정하면 대상 피쳐의 개수, float 형으로 지정하면 전체 피쳐 중 대상 피쳐의 퍼센트임 • 'sqrt'는 전체 피쳐 중 $\sqrt{\text{전체 피쳐 개수}}$ 만큼 선정 • 'auto'로 지정하면 sqrt와 동일 • 'log'는 전체 피쳐 중 $\log_2(\text{전체 피쳐 개수})$ 선정 • 'None'은 전체 피쳐 선정
max_depth	• 트리의 최대 깊이를 규정. • 디폴트는 None, None으로 설정하면 완벽하게 클래스 결정 값이 될 때까지 깊이를 계속 키우며 분할하거나 노드가 가지는 데이터 개수가 min_samples_split보다 작아질 때까지 계속 깊이를 증가시킴. • 깊이가 깊어지면 min_samples_split 설정대로 최대 분할하여 과적합할 수 있으므로 적절한 값으로 제어 필요.
max_leaf_nodes	• 말단 노드(Leaf)의 최대 개수

결정 트리 모델의 시각화

Graphviz 패키지를 사용하여 결정 트리 알고리즘이 어떠한 규칙을 가지고 트리를 생성하는지 시각적으로 볼 수 있음

사이킷런의 트리 모듈은 Graphviz를 이용하기 위해 export_graphviz() 함수를 제공

- `export_graphviz()`는 Graphviz가 읽어 들어서 그래프 형태로 시각화할 수 있는 출력 파일을 생성
- `export_graphviz()`에 인자로 학습이 완료된 estimator, output 파일 명, 결정 클래스의 명칭, 피처의 명칭을 입력



▼ 리프 노드

더 이상 자식 노드가 없는 노드

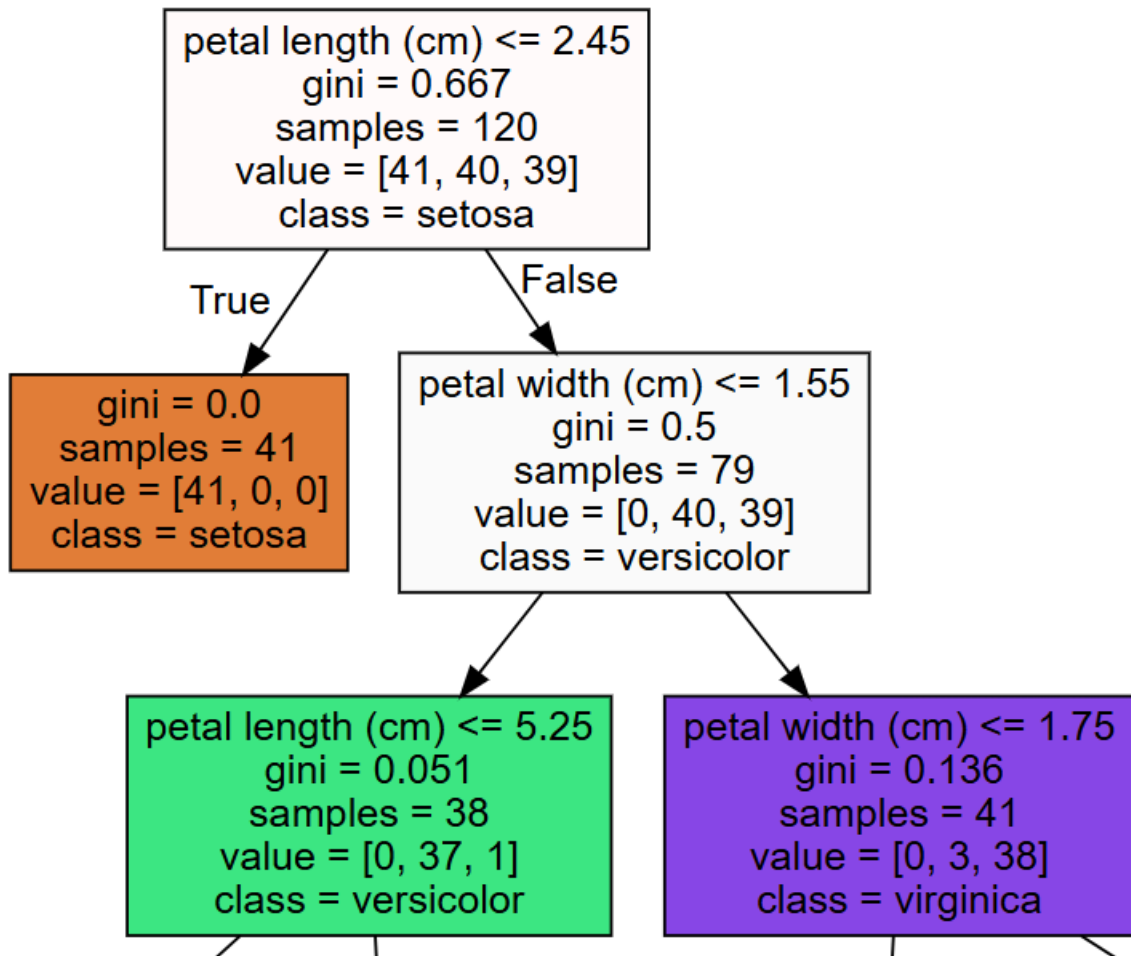
최종 클래스(레이블) 값이 결정되는 노드

리프 노드가 되려면 오직 하나의 클래스 값으로 최종 데이터가 구성되거나 리프 노드가 될 수 있는 하이퍼 파라미터 조건을 충족

▼ 브랜치 노드

자식 노드가 있는 노드

자식 노드를 만들기 위한 분할 규칙 조건을 갖고 있음



▼ 1번 노드(가장 위의 흰색 노드)

`samples = 120` → 전체 데이터가 120개

`value = [41, 40, 39]` → Setosa 41개, Versicolor 40개, Virginica 39개 데이터로 구성

⇒ sample 120개가 `value = [41, 40, 39]` 분포도로 되어 있으므로 지니 계수는 0.667

`petal length(cm) <= 2.45` → 자식 노드 생성 규칙

`class = setosa` → 하위 노드를 가질 경우에 setosa의 개수가 41개로 제일 많다는 의미

`petal length(cm) <= 2.45` 규칙이 True 또는 False로 분기 → 2번, 3번 노드 만들어짐

2번 노드는 모든 데이터가 Setosa로 결정되므로 클래스가 결정된 **리프 노드**가 됨 → 규칙 필요 X

▼ 2번 노드(2번째 줄의 주황 노드)

1번 노드의 petal length(cm) ≤ 2.45 가 True인 규칙으로 생성되는 **리프 노드**

41개의 샘플 데이터 모두 Setosa 이므로 예측 클래스는 Setosa로 결정

지니 계수는 0

▼ 3번 노드(2번째 줄의 흰색 노드)

petal length(cm) ≤ 2.45 가 False인 **규칙 노드**

79개의 샘플 데이터 중 Versicolor 40개, Virginica 39개 → 지니 계수는 0.5로 높음

→ 다음 자식 브랜치 노드로 분기할 규칙이 필요

petal width(cm) ≤ 1.55 → 자식 노드 생성 규칙

▼ 4번 노드(3번째 줄의 초록색 노드)

petal width(cm) ≤ 1.55 가 True인 **규칙 노드**

38개의 샘플 데이터 중 Versicolor 37개, Virginica 1개 → 대부분이 Versicolor

지니 계수는 0.051로 매우 낮음 → 그러나 여전히 두 레이블이 혼재 → petal

width(cm) ≤ 5.25 라는 새로운 규칙으로 다시 자식 노드 생성

▼ 5번 노드(3번째 줄의 보라색 노드)

petal width(cm) ≤ 1.55 가 False인 **규칙 노드**

41개의 샘플 데이터 중 Versicolor 3개, Virginica 38개 → 대부분이 Virginica

지니 계수는 0.136으로 매우 낮음 → 그러나 여전히 두 레이블이 혼재 → petal

width(cm) ≤ 1.75 라는 새로운 규칙으로 다시 자식 노드 생성



각 노드의 색깔은 붓꽃 데이터의 레이블 값을 의미

주황색은 0: Setosa

초록색은 1: Versicolor

보라색은 2: Virginica

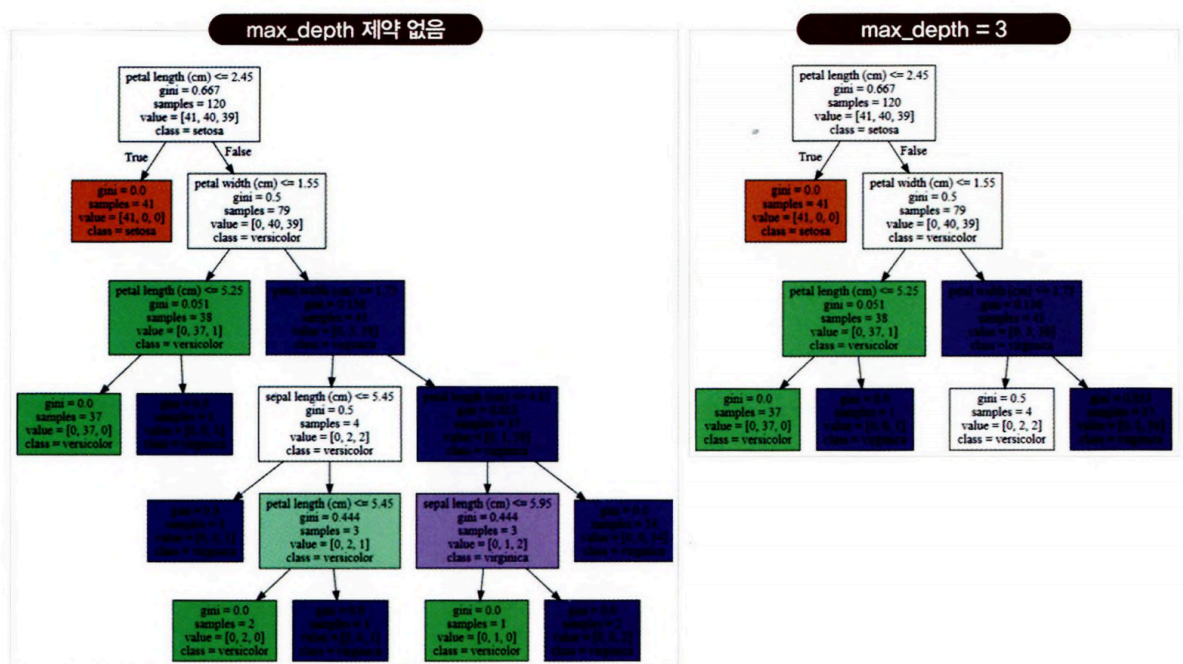
색깔이 짙어질수록 지니 계수가 낮고 해당 레이블에 속하는 샘플 데이터가 많은 의미

결정 트리는 규칙 생성 로직을 미리 제어하지 않으면 완벽하게 클래스 값을 구별해내기 위해 트리 노드를 계속해서 만들 → 매우 복잡한 규칙 트리 → 과적합 문제 발생

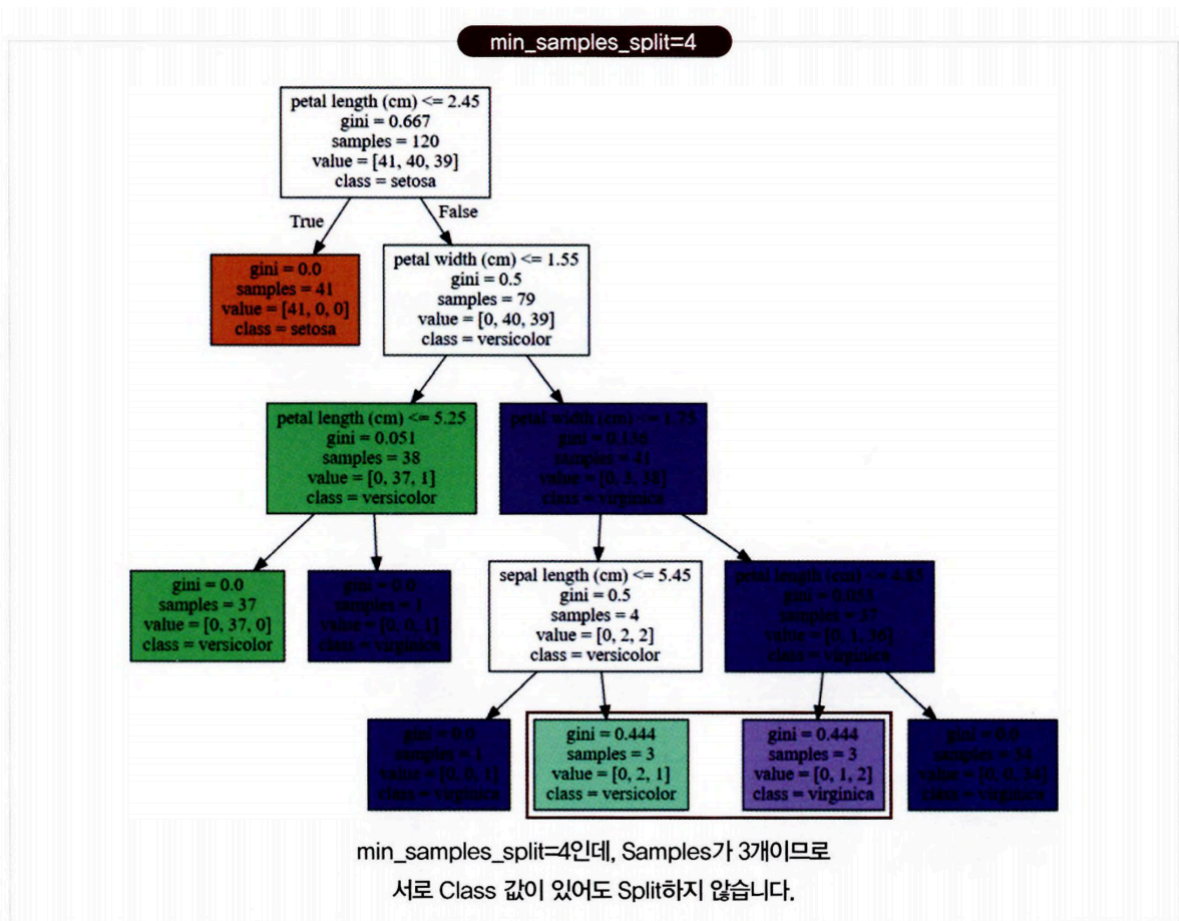
결정 트리는 과적합이 상당히 높은 ML 알고리즘!

⇒ 결정 트리 알고리즘을 제어하는 대부분의 **하이퍼 파라미터**는 복잡한 트리가 생성되는 것을 막기 위한 용도

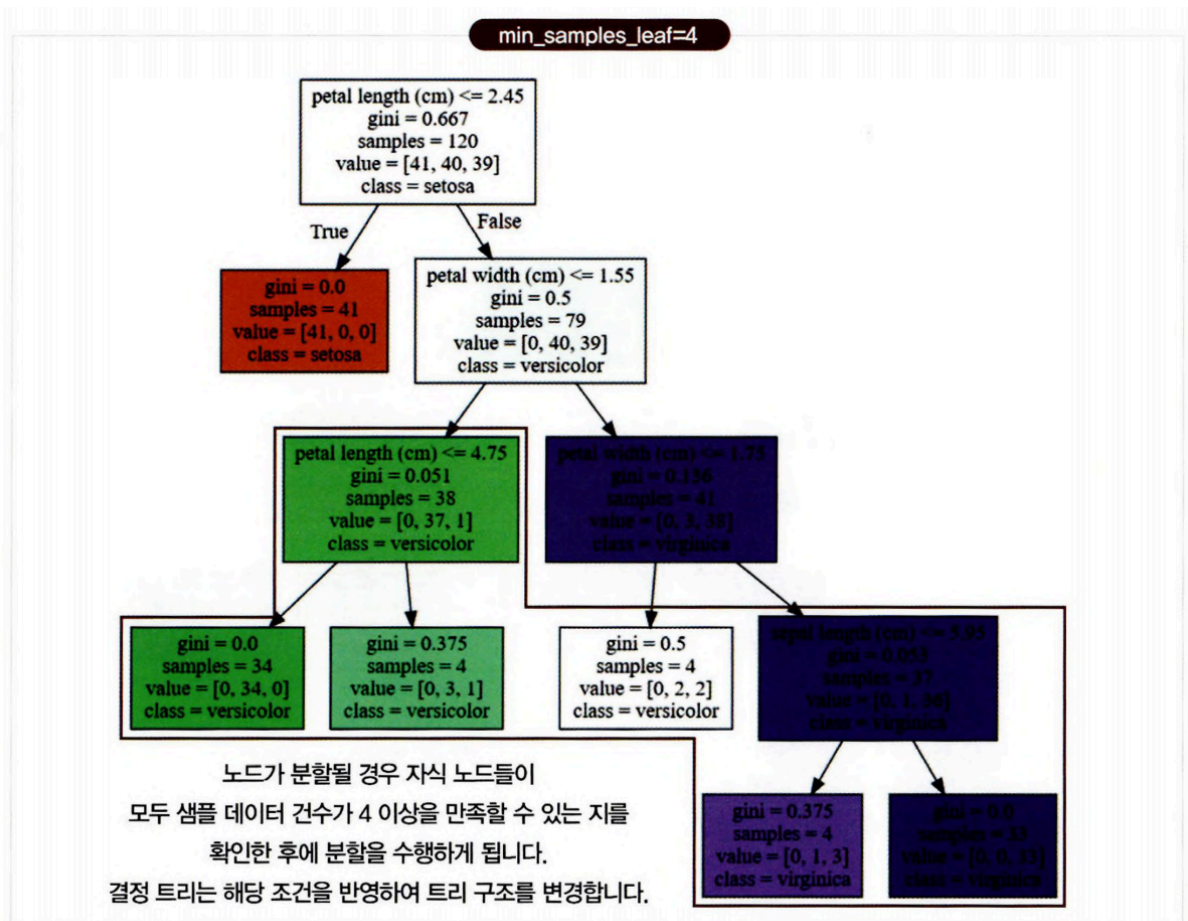
- **max_depth** 하이퍼 파라미터 - 결정 트리의 최대 트리 깊이를 제어



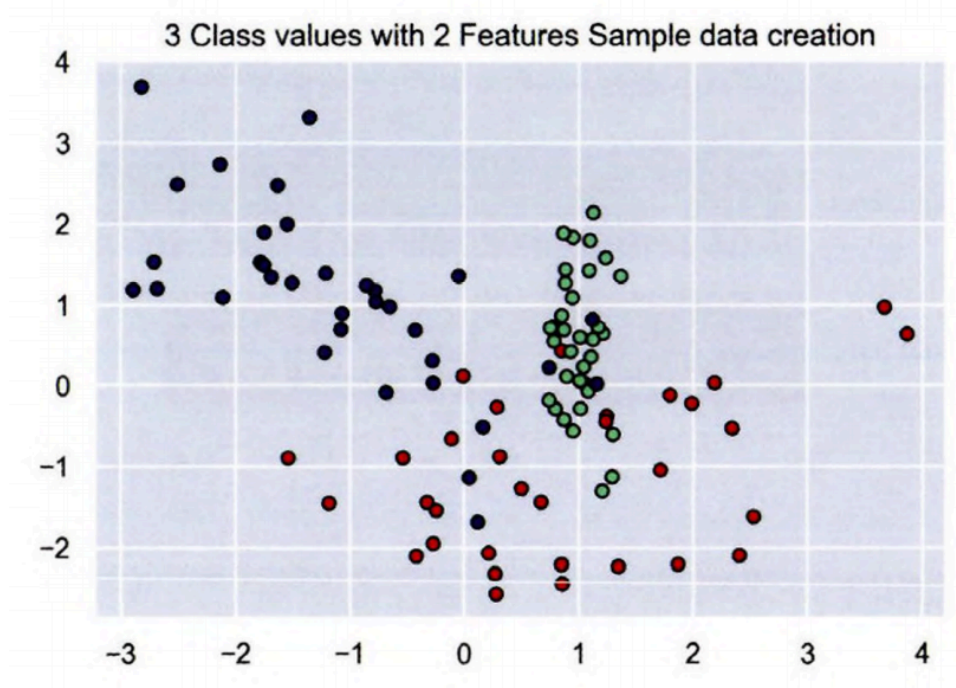
- **min_samples_spilt** 하이퍼 파라미터 - 자식 규칙 노드를 분할해 만들기 위한 최소한의 샘플 데이터 개수



- **min_samples_leaf 하이퍼 파라미터** - 분할될 경우 왼쪽과 오른쪽 자식 노드 각각이 가지게 될 최소 데이터 건수를 지정 → 지정된 최소 데이터 건수보다 더 작은 샘플 데이터 건수를 갖게 된다면, 해당 노드는 더 이상 분할하지 않고 리프 노드가 됨

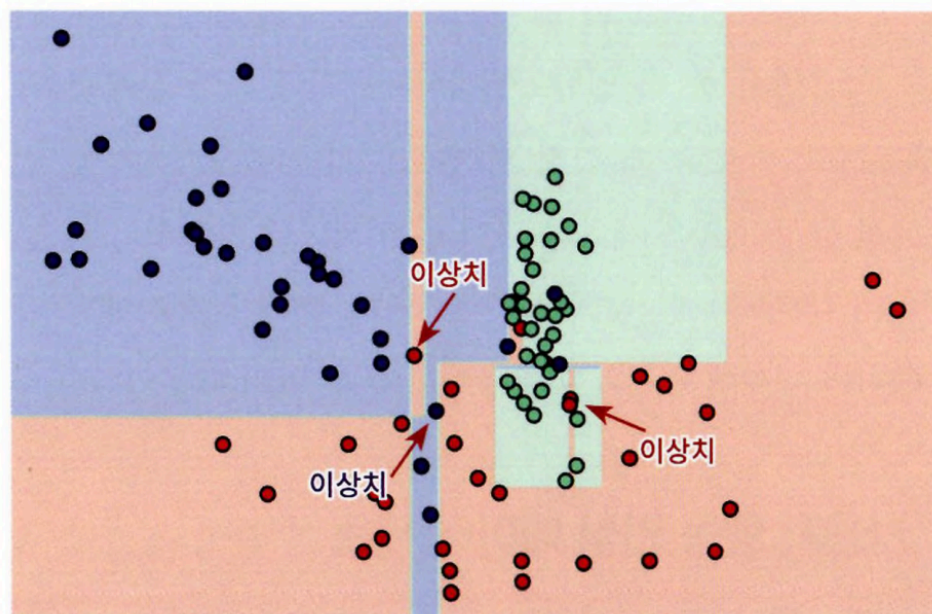


결정 트리 과적합(Overfitting)



↳ 각 피처가 X, Y축으로 나열된 2차원 그래프, 3개의 클래스 값 구분은 색깔로 돼 있음

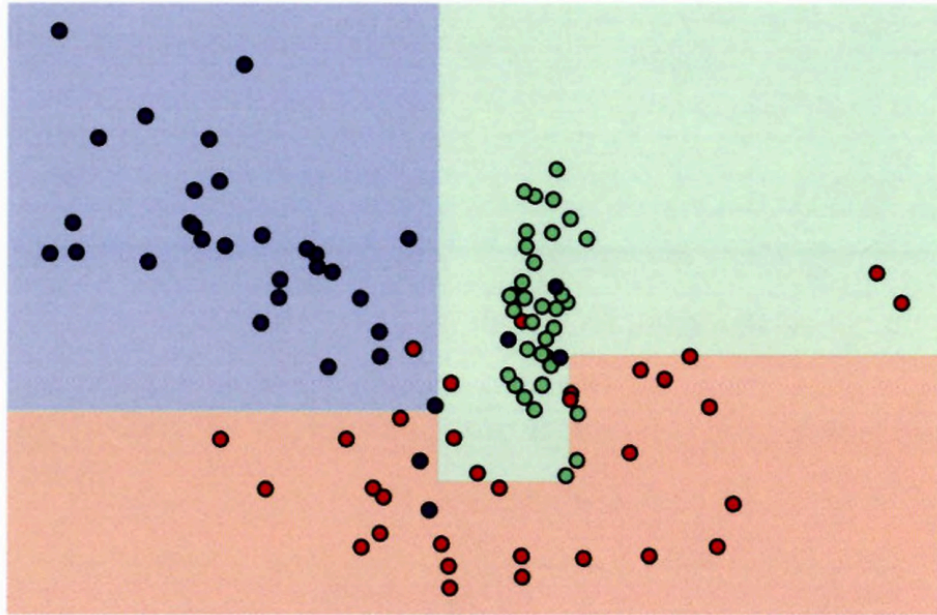
1. 결정 트리 생성에 별다른 제약이 없도록 결정 트리의 하이퍼 파라미터를 디폴트로 한 뒤, 결정 트리 모델이 어떠한 결정 기준을 가지고 분할하면서 데이터를 분류하는지 확인



↳ 일부 이상치(Outlier) 데이터까지 분류하기 위해 분할이 자주 일어나서 결정 기준 경계가 매우 많음

↳ 이렇게 복잡한 모델은 학습 데이터 세트의 특성과 약간만 다른 형태의 데이터 세트를 예측하면 예측 정확도가 떨어짐

2. `min_samples_leaf = 6`을 설정해 6개 이하의 데이터 리프 노드를 생성할 수 있도록 리프 노트 생성 규칙을 완화한 뒤 하이퍼 파라미터를 변경해 어떻게 결정 기준 경계가 변하는지 확인



↳ 이상치에 크게 반응하지 않으면서 좀 더 일반화된 분류 규칙에 따라 분류됐음을 알 수 있음

⇒ 다양한 테스트 데이터 세트를 기반으로 한 결정 트리 모델의 예측 성능은 첫 번째 모델보다는 두 번째 모델이 더 뛰어날 가능성이 높음. - 학습 데이터에만 지나치게 최적화된 분류 기준은 오히려 테스트 데이터 세트에서 정확도를 떨어뜨릴 수 있기 때문

03. 앙상블 학습

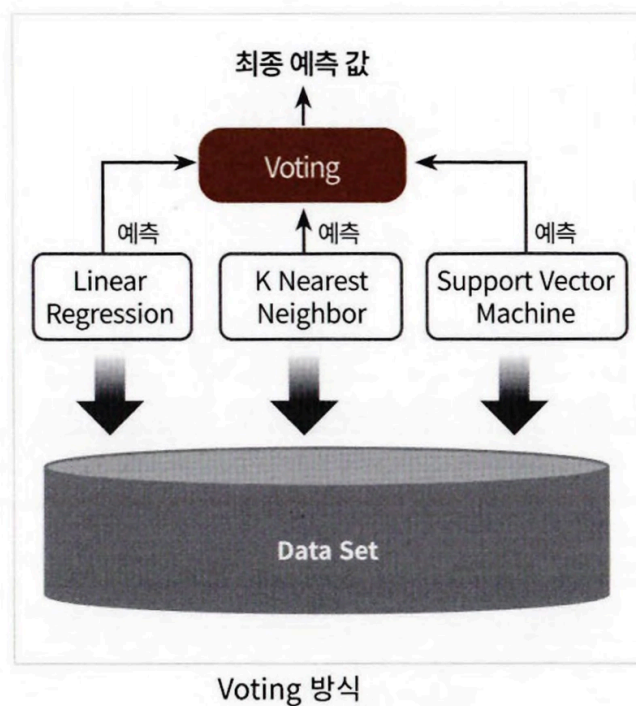
| 앙상블 학습 개요

여러 개의 분류기(Classifier)를 생성하고 그 예측을 결합함으로써 보다 정확한 최종 예측을 도출하는 기법

- 대부분의 정형 데이터 분류 시에는 앙상블이 뛰어난 성능을 나타냄
- XGBoost, LightGBM, 스택킹(Stacking)을 포함해 다양한 유형의 앙상블 알고리즘이 인기
- 앙상블의 학습 유형은 전통적으로 보팅(Voting), 배깅(Bagging), 부스팅(Boosting)의 세 가지로 나눌 수 있음 - 보팅과 배깅은 여러 개의 분류기가 투표를 통해 최종 예측 결과를 결정하는 방식

보팅(Voting)

- 서로 다른 알고리즘을 가진 분류기를 결합

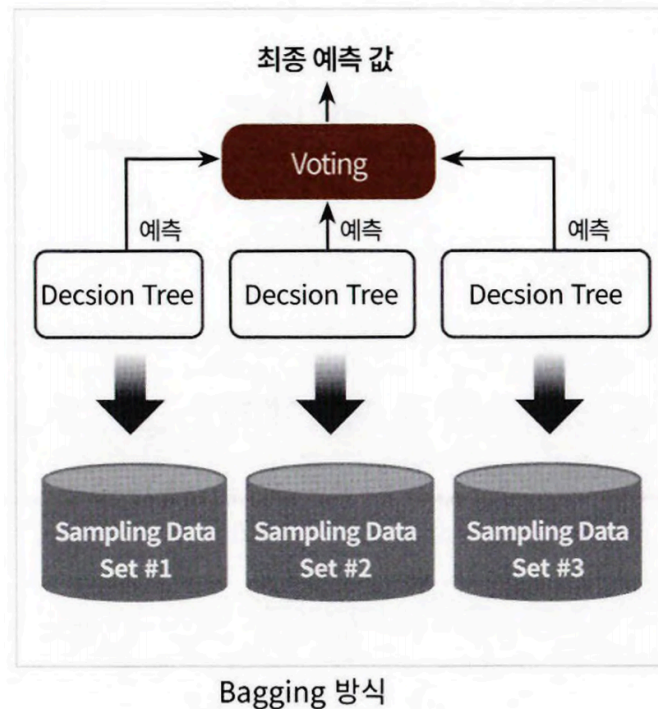


- 선형 회귀, K 최근접 이웃, 서포트 벡터 머신이라는 3개의 ML 알고리즘이 같은 데이터 세트에 대해 학습하고 예측한 결과를 가지고 보팅을 통해 최종 예측 결과를 선정

배깅(Bagging)

- 각각의 분류기가 모두 같은 유형의 알고리즘 기반이지만, 데이터 샘플링을 서로 다르게 가져가면서 학습을 수행해 보팅을 수행하는 것

- 대표적으로 랜덤 포레스트 알고리즘이 있음



- 단일 ML 알고리즘(결정 트리)으로 여러 분류기가 학습으로 개별 예측을 하는데, 학습하는 데이터 세트가 보팅 방식과는 다름
- 개별 분류기에 할당된 학습 데이터는 원본 학습 데이터를 샘플링해 추출하는데, 이렇게 개별 Classifier에게 데이터를 샘플링해서 추출하는 방식을 부트스트래핑 (Bootstrapping) 분할 방식이라고 부름
- 개별 분류기가 부트스트래핑 방식으로 샘플링된 데이터 세트에 대해서 학습을 통해 개별적인 예측을 수행한 결과를 보팅을 통해서 최종 예측 결과를 선정하는 방식이 배깅 앙상블 방식
- 교차 검증이 데이터 세트 간에 중첩을 허용하지 않는 것과 다르게 배깅 방식은 중첩을 허용

부스팅(Boosting)

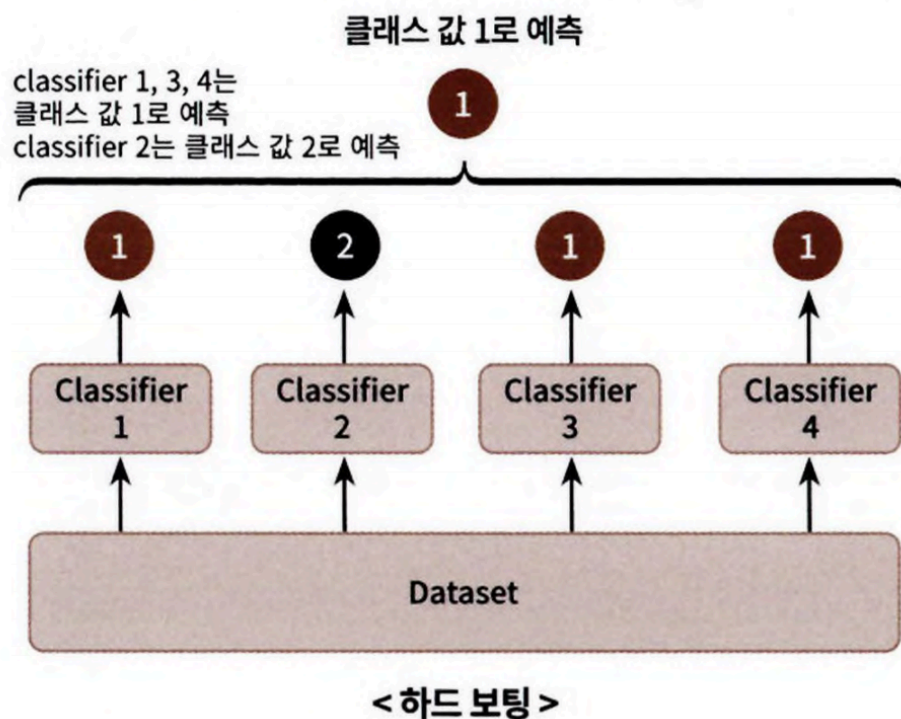
- 여러 개의 분류기가 순차적으로 학습을 수행하되, 앞에서 학습한 분류기가 예측이 틀린 데이터에 대해서는 올바르게 예측할 수 있도록 다음 분류기에게는 가중치를 부여하면서 학습과 예측을 진행
- 예측 성능이 뛰어나 앙상블 학습을 주도, 대표적인 모듈로는 그래디언트 부스트, XGBoost, LightGBM이 있음

보팅 유형 - 하드 보팅(Hard Voting)과 소프트 보팅

하드 보팅(Hard Voting)

- 하드 보팅을 이용한 분류는 다수결 원칙과 비슷, 예측한 결과값들 중 다수의 분류기가 결정한 예측값을 최종 보팅 결과값으로 선정

Hard Voting은 다수의 classifier 간 다수결로 최종 class 결정



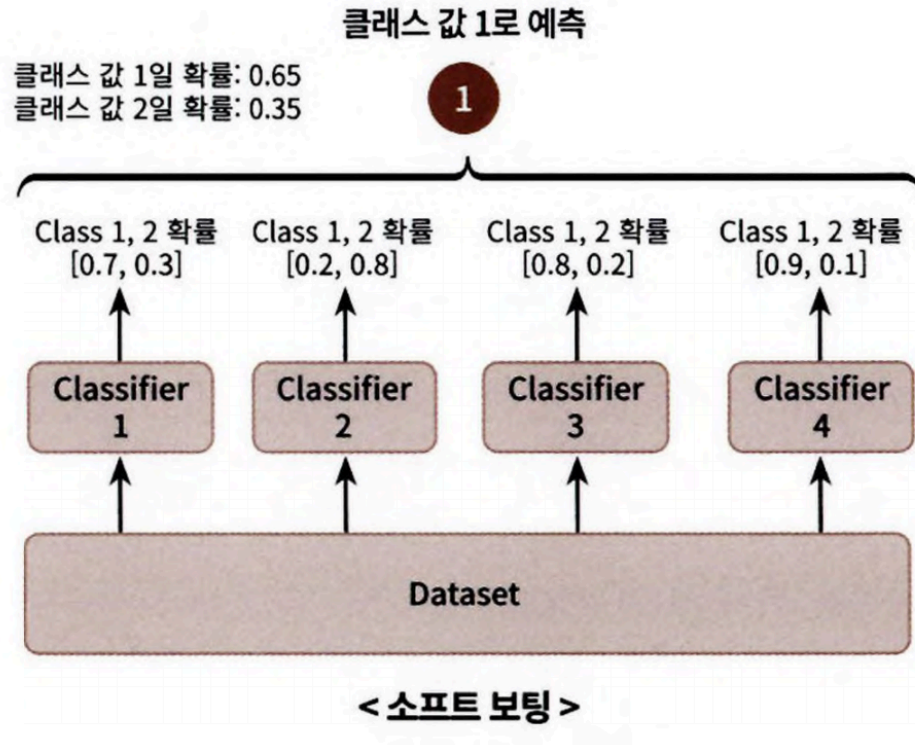
- Classifier 1번, 2번, 3번, 4번인 4개로 구성된 보팅 앙상블 기법에서 분류기 1번, 3번, 4번이 1로 레이블 값을 예측하고 분류기 2번이 2로 레이블을 예측하면 다수결 원칙에 따라서 최종 예측은 레이블 값 1이 됨

소프트 보팅(Soft Voting)

- 분류기들의 레이블 값 결정 확률을 모두 더하고 이를 평균해서 이들 중 확률이 가장 높은 레이블 값을 최종 보팅 결과값으로 선정

- 일반적으로 하드 보팅보다는 소프트 보팅이 예측 성능이 좋아서 더 많이 사용됨

Soft Voting은 다수의 classifier들의 class 확률을 평균하여 결정



- 분류기 1번의 레이블 값 1과 2의 예측 확률이 각각 0.7/0.3이고 분류기 2번은 0.2/0.8, 분류기 3번은 0.8/0.2, 분류기 4번은 0.9/0.1이라면 레이블 값 1의 평균 예측 확률은 분류기 1번, 2번, 3번, 4번의 확률을 모두 더하여 평균하면 0.65가 됨
- 레이블 값 2의 평균 예측 확률도 같은 방법으로 계산하면 0.35가 됨
- 따라서 레이블 값 1의 확률이 더 크므로 레이블 값 1로 최종 보팅하는 것이 소프트 보팅

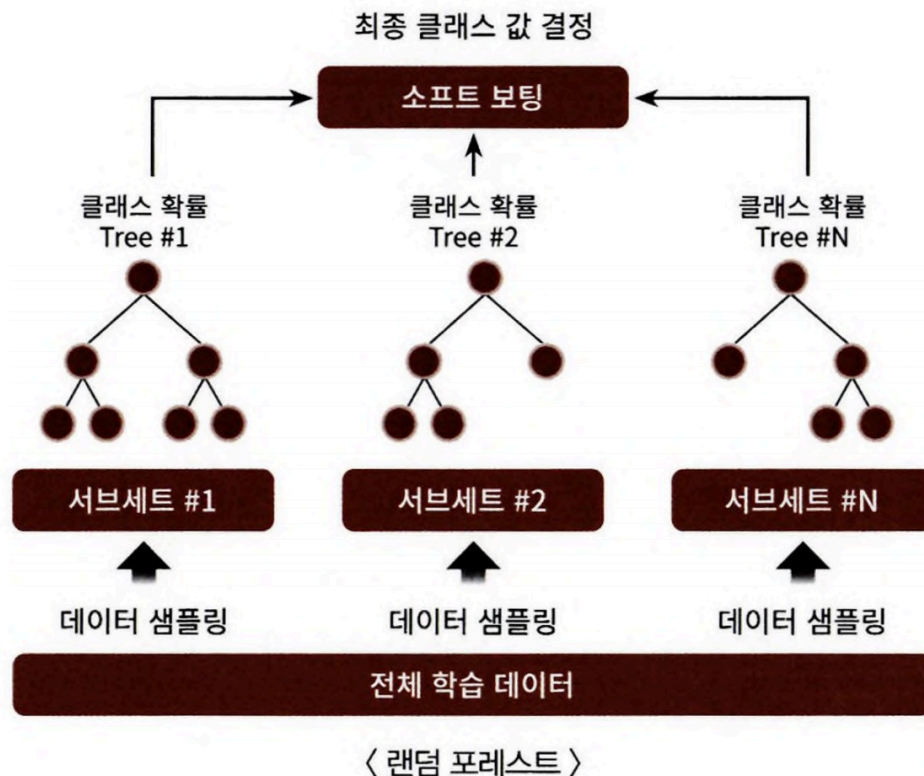
보팅 분류기(Voting Classifier)

사이킷런은 보팅 방식의 앙상블을 구현한 VotingClassifier 클래스를 제공

04. 랜덤 포레스트

랜덤 포레스트의 개요 및 실습

- 앙상블 알고리즘 중 비교적 빠른 수행 속도를 가지고 있고, 다양한 영역에서 높은 예측 성능을 보이고 있음
- 랜덤 포레스트 기반 알고리즘은 결정 트리로서, 결정 트리의 쉽고 직관적인 장점을 그대로 가지고 있음
- 랜덤 포레스트는 여러 개의 결정 트리 분류기가 전체 데이터에서 배깅 방식으로 각자의 데이터를 샘플링해 개별적으로 학습을 수행한 뒤 최종적으로 모든 분류기가 보팅을 통해 예측 결정을 하게 됨

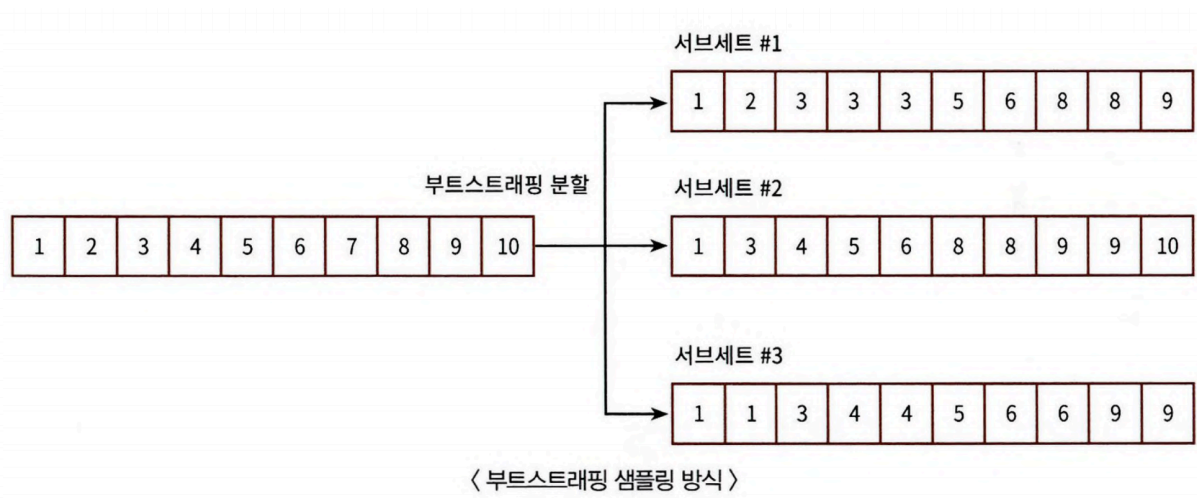


- 랜덤 포레스트는 개별적인 분류기의 기반 알고리즘은 결정 트리이지만 개별 트리가 학습하는 데이터 세트는 전체 데이터에서 일부가 중복되게 샘플링된 데이터 세트

? 부트스트래핑(Bootstrapping)이란...?
여러 개의 데이터 세트를 중복되게 분리하는 것

원래 부트스트랩은 통계학에서 여러 개의 작은 데이터 세트를 임의로 만들어 개별 평균의 분포도를 측정하는 등의 목적을 위한 샘플링 방식을 지칭
랜덤 포레스트의 서브세트(subset) 데이터는 이러한 부트스트래핑으로 데이터가 임의로 만들어짐

- 서브세트의 데이터 건수는 전체 데이터 건수와 동일하지만, 개별 데이터가 중복되어 만들어짐
- 원본 데이터의 건수가 10개인 학습 데이터 세트에 랜덤 포레스트를 3개의 결정 트리 기반으로 학습하려고 `n_estimators=3`으로 하이퍼 파라미터를 부여하면 다음과 같이 데이터 서브세트가 만들어짐



- 사이킷런은 `RandomForestClassifier` 클래스를 통해 랜덤 포레스트 기반의 분류를 제공

랜덤 포레스트 하이퍼 파라미터 및 튜닝

트리 기반의 앙상블 알고리즘의 단점은 하이퍼 파라미터가 너무 많고, 그로 인해서 튜닝을 위한 시간이 많이 소모된다는 점

많은 시간을 소모했음에도 튜닝 후 예측 성능이 크게 향상되는 경우가 많지 않음

<파라미터>

- `n_estimators`: 랜덤 포레스트에서 결정 트리의 개수를 지정. 디폴트는 10개. 많이 설정할수록 좋은 성능을 기대할 수 있지만 계속 증가시킨다고 성능이 무조건 향상되는 것은 아님. 또한 늘릴수록 학습 수행 시간이 오래 걸리는 것도 감안
- `max_features`: 결정 트리에 사용된 `max_features` 파라미터와 동일. 하지만 `RandomForestClassifier`의 기본 `max_features`는 'None'이 아니라 'auto'. 즉 'sqrt'와 같음. 따라서 랜덤 포레스트의 트리를 분할하는 피처를 참조할 때 전체 피처가 아니라 sqrt(전체 피처 개수)만큼 참조
- `max_depth`, `min_samples_leaf`, `min_samples_split`: 결정 트리에서 과적합을 개선하기 위해 사용되는 파라미터가 랜덤 포레스트에도 똑같이 적용 가능